

KebabRank Admin Management Guide

Table of Contents

1. [Quick Command Reference](#)
 2. [Daily Operations](#)
 3. [City Management](#)
 4. [Data Updates & Rankings](#)
 5. [AI Analysis Management](#)
 6. [Troubleshooting](#)
 7. [Server vs Local Management](#)
 8. [Database Direct Access](#)
-

Quick Command Reference

Most Used Commands

```
bash
```

```
# Add a new city
```

```
python add_city.py "City Name"
```

```
# Update rankings for a specific city
```

```
python update_single_city.py "City Name"
```

```
# Fix all rankings (when order is wrong)
```

```
python fix_rankings.py
```

```
# Run AI analysis for a city
```

```
python update_ai_analysis.py "City Name" --limit 10
```

```
# Check system health
```

```
python debug_system.py
```

Daily Operations

Morning Checklist (5 minutes)

1. **Check website is live:** Visit kebabrank.com

2. **Check last update time:** Look at countdown timer
3. **Monitor any errors:** Check server logs

Weekly Tasks (30 minutes)

1. **Review new city requests:** Check emails/social media
 2. **Add new cities:** Use `add_city.py`
 3. **Update AI analysis:** For top 5 cities
 4. **Generate sitemap:** `python generate_sitemap.py`
-



City Management

Adding a New City

```
bash

# Step 1: Add city to database
python add_city.py "Nowy Sącz"

# Step 2: Verify it was added
python list_cities.py

# Step 3: Update sitemap
python generate_sitemap.py
```

Removing a City

```
sql

-- In Supabase SQL editor
DELETE FROM cities WHERE name = 'City Name';
```

Updating City Data

```
bash

# Update just one city (faster)
python update_single_city.py "Kraków"

# Update all cities (takes ~30 minutes)
python update_all_cities.py
```

Data Updates & Rankings

Fix Ranking Issues

Problem: Rankings show wrong order after running fix_rankings.py

Solution: The issue is that rankings are cached. Follow these steps:

```
bash
```

```
# Step 1: Run fix rankings
```

```
python fix_rankings.py
```

```
# Step 2: Clear cache and force refresh
```

```
python force_refresh.py "City Name"
```

```
# Step 3: If still not working, update the city data
```

```
python update_single_city.py "City Name"
```

Create force_refresh.py:

python

force_refresh.py

```
import sys
from datetime import datetime
from dotenv import load_dotenv
from services.database import DatabaseService
from services.data_updater import DataUpdater
from services.google_places import GooglePlacesService
from services.ranking import RankingService

load_dotenv()

def force_refresh_city(city_name):
    """Force refresh rankings for a city"""
    print(f"🔄 Force refreshing {city_name}...")

    db_service = DatabaseService(os.getenv('SUPABASE_URL'), os.getenv('SUPABASE_KEY'))
    google_service = GooglePlacesService(os.getenv('GOOGLE_API_KEY'))
    ranking_service = RankingService()
    data_updater = DataUpdater(google_service, db_service, ranking_service)

    # Force update
    data_updater.update_city_data(city_name)

    # Recalculate rankings
    city_id = db_service.get_city_id(city_name)
    if city_id:
        db_service.recalculate_city_rankings(city_id)
        print(f"✅ {city_name} refreshed successfully!")
    else:
        print(f"❌ City {city_name} not found!")

if __name__ == "__main__":
    if len(sys.argv) > 1:
        city_name = ' '.join(sys.argv[1:])
        force_refresh_city(city_name)
    else:
        print("Usage: python force_refresh.py [City Name]")
```

Manual Ranking Update

```
bash
```

```
# Check current rankings
```

```
python check_rankings.py "Kraków"
```

```
# Fix specific city
```

```
python fix_city_rankings.py "Kraków"
```

```
# Update all rankings
```

```
python update_all_rankings.py
```



AI Analysis Management

Running AI Analysis

For Testing (Local)

```
bash
```

```
# Single city, few kebabs
```

```
python update_ai_analysis.py "Kraków" --limit 3
```

For Production (Server)

```
bash
```

```
# Major cities (10 kebabs each)
```

```
python batch_ai_update.py
```

```
# Choose option 1
```

```
# All cities (different limits)
```

```
python update_all_ai.py
```

AI Cost Management

- **Monitor usage:** ~\$0.002 per kebab
 - **Monthly budget:** ~\$50 for all cities
 - **Optimize by:** Analyzing only top 10 per city
-



Troubleshooting

Common Issues & Solutions

1. Rankings Not Updating

```
bash

# Check if data is fresh
python check_data_age.py "City Name"

# Force update if stale
python force_update.py "City Name"
```

2. AI Analysis Not Showing

```
bash

# Check if AI data exists
python check_ai_data.py "City Name"

# Re-run AI analysis
python update_ai_analysis.py "City Name" --limit 5
```

3. Website Not Loading New Data

```
bash

# On server
cd /opt/kebab-rank
docker-compose -f docker-compose.prod.yml restart

# Clear Flask cache
rm -rf __pycache__
rm -rf services/__pycache__
```



Server vs Local Management

Local Development

```
bash
```

```
# Always activate virtual environment first
```

```
venv\Scripts\activate # Windows
```

```
source venv/bin/activate # Mac/Linux
```

```
# Test changes locally
```

```
python app.py
```

```
# Run scripts
```

```
python script_name.py
```

Server Management

```
bash
```

```
# SSH to server
```

```
ssh user@your-server-ip
```

```
# Navigate to project
```

```
cd /opt/kebab-rank
```

```
# Run scripts (no venv needed in Docker)
```

```
docker exec -it kebab-rank-app python script_name.py
```

```
# View Logs
```

```
docker logs kebab-rank-app
```

```
# Restart after changes
```

```
docker-compose -f docker-compose.prod.yml restart
```



Database Direct Access

Quick Supabase SQL Queries

Check Rankings for a City

sql

-- Get current rankings for Kraków

SELECT

kp.name,
rh.rating,
rh.total_reviews,
rh.rank_score,
rh.city_rank,
rh.data_fetched_at

FROM kebab_places kp

JOIN ratings_history rh ON kp.id = rh.kebab_place_id

WHERE kp.city_id = (SELECT id FROM cities WHERE name = 'Kraków')

AND rh.data_fetched_at = (

SELECT MAX(data_fetched_at)

FROM ratings_history

WHERE kebab_place_id = kp.id

)

ORDER BY rh.rank_score DESC;

Fix Specific Ranking

sql

-- Manually update rank for a kebab

UPDATE ratings_history

SET city_rank = 1

WHERE kebab_place_id = (

SELECT id FROM kebab_places

WHERE name = 'Kebab u Rudego'

);

Check AI Analysis Status

sql

-- See which cities have AI analysis

```
SELECT
    c.name as city,
    COUNT(DISTINCT kp.id) as total_kebabs,
    COUNT(DISTINCT aa.kebab_place_id) as analyzed_kebabs
FROM cities c
LEFT JOIN kebab_places kp ON c.id = kp.city_id
LEFT JOIN ai_analysis aa ON kp.id = aa.kebab_place_id
GROUP BY c.id, c.name
ORDER BY c.name;
```



Create Admin Dashboard Script

Save this as `admin.py`:

python

```
#!/usr/bin/env python
```

```
import os
import sys
from datetime import datetime
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
def show_menu():
    """Display admin menu"""
    print("\n" + "="*50)
    print("🍴 KebabRank Admin Panel")
    print("="*50)
    print("1. City Management")
    print("2. Update Rankings")
    print("3. AI Analysis")
    print("4. System Health Check")
    print("5. Database Queries")
    print("0. Exit")
    print("="*50)
```

```
def city_management():
    """City management submenu"""
    print("\n--- City Management ---")
    print("1. Add new city")
    print("2. List all cities")
    print("3. Update city data")
    print("4. Remove city")
    print("0. Back")

    choice = input("\nChoice: ")

    if choice == "1":
        city = input("Enter city name: ")
        os.system(f"python add_city.py {city}")
    elif choice == "2":
        os.system("python list_cities.py")
    elif choice == "3":
        city = input("Enter city name: ")
        os.system(f"python update_single_city.py {city}")
    # ... etc
```

```
def main():
```

```

"""Main admin interface"""
while True:
    show_menu()
    choice = input("\nEnter choice: ")

    if choice == "0":
        print("Goodbye!")
        break
    elif choice == "1":
        city_management()
    elif choice == "2":
        os.system("python fix_rankings.py")
    elif choice == "3":
        city = input("Enter city name (or 'all'): ")
        if city.lower() == 'all':
            os.system("python update_ai_analysis.py --all")
        else:
            os.system(f"python update_ai_analysis.py {city}")
    elif choice == "4":
        os.system("python debug_system.py")
    else:
        print("Invalid choice!")

    input("\nPress Enter to continue...")

if __name__ == "__main__":
    main()

```

Best Practices

1. Always test locally first

```

bash

python script.py --test

```

2. Backup before major changes

```

bash

python backup_data.py

```

3. Monitor API usage

- Google Places: \$200 free monthly

- OpenAI: Check dashboard daily

4. Update documentation

- Log all changes
- Update this guide when adding features

5. Regular maintenance

- Weekly: Update top 10 cities
- Monthly: Update all cities
- Quarterly: Full AI re-analysis

Emergency Procedures

Website Down

```
bash
```

```
# 1. SSH to server
```

```
ssh user@server
```

```
# 2. Check Docker status
```

```
docker ps
```

```
# 3. Restart services
```

```
docker-compose -f docker-compose.prod.yml down
```

```
docker-compose -f docker-compose.prod.yml up -d
```

```
# 4. Check Logs
```

```
docker logs kebab-rank-app --tail 100
```

Data Corruption

```
bash
```

```
# 1. Stop updates
```

```
touch MAINTENANCE_MODE
```

```
# 2. Backup current data
```

```
python emergency_backup.py
```

```
# 3. Restore from last good backup
```

```
python restore_backup.py --date 2024-01-19
```

```
# 4. Resume operations
```

```
rm MAINTENANCE_MODE
```

Quick Contacts

- **Supabase Dashboard:** [your-project].supabase.co
 - **Server SSH:** ssh user@your-server
 - **Domain:** Hostinger control panel
 - **Monitoring:** uptimerobot.com (set this up!)
-

Remember: Always test locally, backup regularly, and document changes!